Mitali Bhatia
D15C 16
MLDL

Experiment No 09

Aim: Implement Recurrent Neural Network (RNN) / LSTM for time series or text data

1. Dataset Source

The dataset used in this experiment is **Netflix Stock Price Dataset**.

Source: Kaggle
Dataset: Netflix Stock Data
Link: https://www.kaggle.com/datasets/adilshamim8/netflix-stock-price-history

This dataset contains historical stock prices of Netflix used for time series prediction.

---

2. Dataset Description

The dataset contains historical stock price information of Netflix over time.

Features in dataset

- Date — Trading date
- Open — Opening price
- High — Highest price of day
- Low — Lowest price of day
- Close — Closing price
- Volume — Number of shares traded

Target Variable

Close price is used as the target variable for prediction.

Dataset Characteristics

- Time series dataset
- Numerical values
- Ordered by date
- Used for sequence prediction
- No categorical variables

Dataset Size

- Rows: Historical daily stock records
- Columns: 6
- Input used: Close price only

---

3. Mathematical Formulation of LSTM

LSTM is a type of Recurrent Neural Network (RNN) used for sequence prediction.

LSTM uses memory cells and three gates:

- Forget Gate
- Input Gate
- Output Gate

Forget Gate

Decides what information to remove

$$\boxed{f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)}$$

Input Gate

Decides what new information to store

$$\boxed{i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)}$$

Candidate memory:

$$\boxed{\tilde{C}_t = tanh(W_c[h_{t-1}, x_t] + b_c)}$$

Cell State Update

$$\boxed{C_t = f_t \cdot C_{t-1} + i_t \cdot \tilde{C}_t}$$

Output Gate:

$$\boxed{o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)}$$

Final Output:

$$\boxed{h_t = o_t \cdot tanh(C_t)}$$

---

4. Algorithm Limitations

- Requires large dataset for better prediction
- Training takes more time
- Sensitive to hyperparameters
- Difficult to interpret model
- Works only for sequential data
- Overfitting possible if epochs too high

---

5. Methodology / Workflow

Step 1: Upload Dataset

- Dataset uploaded in Google Colab

Step 2: Import Libraries

- NumPy, Pandas, Matplotlib
- Keras for LSTM
- MinMaxScaler for normalization

Step 3: Load Dataset

- CSV file loaded using pandas

Step 4: Select Data

- Only “Close” column selected

Step 5: Normalize Data

- Data scaled between 0 and 1

Step 6: Create Sequences

- 3 previous values used to predict next value

Example:
Input → [100, 105, 110]
Output → 115

Step 7: Reshape Data

- Converted into LSTM format

Step 8: Build Model

- LSTM layer with 50 neurons
- Dense output layer

Step 9: Train Model

- Trained for 50 epochs

Step 10: Prediction

- Last 3 values used to predict next price

Step 11: Visualization

- Graph plotted for actual prices

---

6. Performance Analysis

Model performance is analyzed using:

Loss Function

Mean Squared Error (MSE) is used

Lower loss means better prediction.

Prediction Output

Model predicts next closing price based on last 3 values.

Graph Analysis

The graph shows:

- Actual stock price trend
- Model learns price movement
- Prediction follows time pattern

Interpretation

The LSTM model successfully learns sequential pattern of stock price and predicts future value.

---

7. Hyperparameter Tuning

Parameters Used:

- Epochs = 50
- LSTM Units = 50
- Time Steps = 3
- Optimizer = Adam

Tuning Methods:

- Increase epochs → better learning
- Increase time steps → better pattern understanding
- Change number of neurons
- Try different optimizers

Impact:

- Proper tuning improves accuracy
- Wrong tuning can cause poor results

CODE:

```
# Step 1: Upload Dataset
from google.colab import files
uploaded = files.upload()

# Step 2: Import Libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```python
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense
from sklearn.preprocessing import MinMaxScaler

# Step 3: Load Dataset
df = pd.read_csv("Netflix_stock_data.csv")

print(df.head())

# Step 4: Use Closing Price
data = df['Close'].values
data = data.reshape(-1, 1)

# Step 5: Normalize Data
scaler = MinMaxScaler()
data = scaler.fit_transform(data)

# Step 6: Create Dataset (3 time steps)
X = []
y = []

for i in range(len(data)-3):
    X.append(data[i:i+3])
    y.append(data[i+3])

X = np.array(X)
y = np.array(y)

# Step 7: Reshape for LSTM
X = X.reshape((X.shape[0], X.shape[1], 1))

# Step 8: Build LSTM Model
model = Sequential()

model.add(LSTM(50, activation='relu', input_shape=(3,1)))
model.add(Dense(1))
```

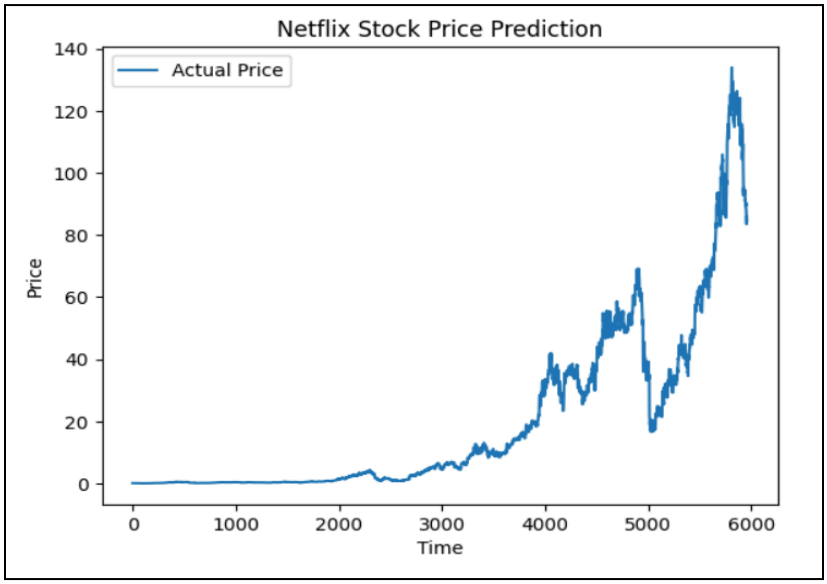
```python
model.compile(optimizer='adam', loss='mse')

# Step 9: Train Model
model.fit(X, y, epochs=50, verbose=1)

# Step 10: Predict Next Value
test_input = data[-3:]
test_input = test_input.reshape((1, 3, 1))

prediction = model.predict(test_input)
prediction = scaler.inverse_transform(prediction)

print("Predicted next closing price:", prediction)

# Step 11: Plot Graph
plt.plot(scaler.inverse_transform(data), label="Actual Price")
plt.title("Netflix Stock Price Prediction")
plt.xlabel("Time")
plt.ylabel("Price")
plt.legend()
plt.show()
```

OUTPUT:

	Date	Close	High	Low	Open	Volume
0	2002-05-23	0.119643	0.124286	0.114571	0.115643	1047900000
1	2002-05-24	0.121000	0.122500	0.119714	0.121429	111048000
2	2002-05-28	0.115714	0.123214	0.115714	0.121357	66094000
3	2002-05-29	0.110357	0.116429	0.108571	0.116429	67578000
4	2002-05-30	0.107143	0.110786	0.107143	0.110786	101542000

```
Epoch 1/50
/usr/local/lib/python3.12/dist-packages/keras/src/layers/rnn/
  super().__init__(**kwargs)
187/187 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.0172
Epoch 2/50
187/187 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 7.5638e-05
Epoch 3/50
187/187 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 7.0366e-05
Epoch 4/50
187/187 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 6.7003e-05
```

```
Epoch 41/50
187/187 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - loss: 6.1166e-05
Epoch 42/50
187/187 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 5.4055e-05
Epoch 43/50
187/187 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 5.4811e-05
Epoch 44/50
187/187 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 5.4740e-05
Epoch 45/50
187/187 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 5.5980e-05
Epoch 46/50
187/187 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - loss: 5.4809e-05
Epoch 47/50
187/187 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - loss: 5.5349e-05
Epoch 48/50
187/187 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - loss: 5.4188e-05
Epoch 49/50
187/187 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - loss: 5.5746e-05
Epoch 50/50
187/187 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 5.2573e-05
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 183ms/step
Predicted next closing price: [[85.03075]]
```



Conclusion:

In this experiment, an LSTM model was implemented to predict Netflix stock closing prices using historical data. The dataset was preprocessed by selecting the Close price and normalizing the values. Time step sequences were created and fed into the LSTM network for training. After training, the model successfully predicted the next stock price based on previous values.

The results show that LSTM can effectively learn patterns from time series data and make future predictions. The model captured the trend of stock prices and produced reasonable output. Therefore, LSTM is suitable for stock price forecasting and other sequential prediction problems.